

Phase 8 Report

Kubernetes Workload Analyzer — scripted demo, final demonstration

Phase status	DONE (2026-09-26)
Report date	2026-09-26
Repository	github.com/ISHANT57/kubernetes-workload-analyzer
State of main	a998b5d , merged as 4c2d200 (PR #13)
All 8 phases	DONE
Written for	the project owner, as a study and reference document

Evidence labels used throughout. **MEASURED** a command was run on this machine and the number is its output · **VERIFIED** checked against upstream documentation, source or a registry · **ESTIMATED** a projection, not measured · **UNVERIFIED** not yet tested; a test is named.

Contents

1. What Phase 8 was for
2. The runbook's structure
3. A real observation caught while writing the runbook
4. What was deliberately not fabricated
5. Git and GitHub workflow
6. Follow-up artifacts added after Phase 8 closed
7. Honest project status against the original brief

1. What Phase 8 was for

Phases 0-7 built and verified the platform piece by piece. Phase 8's job was to prove the whole thing end to end against the original brief's own success criteria (§24, items 1-8 — the demonstrable core; items 9-14 are documentation/testing artifacts covered by existing docs and the test suite, not a live demo step) and write that proof down as a runbook the owner can re-run. Done condition (PHASES.md): "Demo runbook in docs/runbook.md ."

2. The runbook's structure

`docs/runbook.md` has one section per success criterion, each with a real command run against the live in-cluster analyzer:

1. A Kubernetes cluster being monitored — `/healthz`, `/readyz`, `/api/runs/latest`
2. Workloads discovered automatically — live deployment count vs. `workloads_seen`
3. CPU/memory metrics collected — `/api/timeseries`
4. Resource requests vs. actual usage — R001/R002 evidence
5. Pod failures/restarts detected — R003/R004 on the `oom/crashloop` fixtures
6. Health issues on the dashboard — the five pages, Playwright-verified in Phase 5/6
7. Resource optimization opportunities — the negative controls (`right-sized`, `cpu-spike`, `hpa-coupled`) and why each is excluded
8. Estimated cost impact with documented assumptions — the pricing formula and disclaimer

Plus a bonus "Failure-path demo" reproducing Phase 7's real Prometheus-outage test.

3. A real observation caught while writing the runbook

Running §7's command right after Phase 7's deliberate Prometheus outage test showed every R001/R002 candidate at `data_quality: insufficient` ("less than 80% coverage over the minimum 30m0s window") instead of firing — confirmed with `cmd/verify` against the live fixtures. The outage itself had just created a real gap in that 30-minute coverage window. This is R005 (the data-quality gate) working exactly as designed: it declines to assert a resource finding on thin data rather than guessing through the gap, and self-clears once the gap ages out of the lookback. Documented live in the runbook rather than smoothed over with a normal-looking example.

4. What was deliberately not fabricated

An early draft of the runbook's cost-impact section included example `allocation_cost_usd/optimized_cost_usd` numbers that were not actually pulled from a live API response — caught and corrected before merging, per `AGENTS.md`'s "never invent metrics" rule. The corrected section instead shows the real evidence captured live from `cmd/verify` and works the real cost formula (`internal/findings/findings.go`, `CostHours = 730`) through it by hand, labelled explicitly as a derived illustration rather than a copied API response, since R001 was not firing (§3) at the moment of writing.

5. Git and GitHub workflow

Branch `phase-8` → commit `a998b5d` ("feat(phase-8): scripted demo runbook, final demonstration") → PR #13 → merged as `4c2d200` → branch deleted. All 8 phases in `PHASES.md` are DONE as of this merge.

6. Follow-up artifacts added after Phase 8 closed

Three gaps against the original brief's full documentation/testing ambition (identified in an honest completion review, not part of the Phase 0-8 plan itself) were closed afterward:

- `docs/performance.md` — real measured API latency percentiles, resource usage under load, and analysis-run duration from the live in-cluster analyzer's own `/metrics`.
- `backend/cmd/loadtest/main.go` — a small `stdlib`-only concurrent load generator (brief §17's "basic load tests"), used to produce the numbers in `docs/performance.md`.
- This report and `phase-3` through `phase-7-report.{html,pdf}` — the PDF write-up convention established in Phases 1-2 had lapsed for Phases 3-8; backfilled here from the already-verified facts in `PHASES.md` and `docs/*`, not reconstructed from memory.

7. Honest project status against the original brief

The demoable MVP (brief §24 items 1-8, PHASES.md Phases 0-8) is complete and live-verified. Explicitly out of scope by the brief's own written decision (docs/requirements.md , "Out of MVP"): authentication beyond local access, multi-cluster, alerting, finding history/persistence, an AI explainability layer, and CI/CD deployment. Also not built: a formal KRR/OpenCost comparison (deferred, Phase 4), the Pending/unschedulable health rule and R004's near-limit-memory variant (both named backlog items), and docs/observability.md as a dedicated file (its content is covered across docs/architecture.md , docs/performance.md and this report series instead). None of these were silently dropped — each is named in a doc.